

Custom Resource Definitions

Extending the Kubernetes API

Intermediate Kubernetes — Extending Kubernetes (companion to Module 13)

Objectives

- Explain how a **CRD** adds a new resource type to the API
- Manage **custom resources** with the same `kubectl` verbs as built-ins
- Enforce structure with an **OpenAPI validation schema**
- Make a type discoverable (`api-resources`, `explain`, printer columns, short names)
- Understand the **operator pattern**: CRD (API) + controller (behavior)

Why Extend the API?

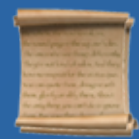
You already rely on it

- Flux — `HelmRelease`, `Kustomization`
- cert-manager — `Certificate`
- Gateway API — `HTTPRoute`
- Kyverno — `ClusterPolicy`

What you gain

- A domain object that *speaks your problem*
- Declarative, GitOps-friendly config
- RBAC, audit, `kubectl` — all for free

A CRD lets the cluster store and serve *your* concepts as first-class API objects.



The CustomResourceDefinition

Teach the API server a new kind

CRD Anatomy

Identity

- `group + versions` (e.g. `v1`)
- `names: plural, singular, kind, shortNames`
- `scope: Namespaced or Cluster`

Shape

- `schema.openapiV3Schema` — validation
- `additionalPrinterColumns` — `kubectl get columns`
- `subresources` — `status, scale`

Name is always `<plural>.<group>`. Once Established, the type appears in `kubectl api-resources`.



Custom Resources & Validation

A **custom resource** is an instance of your type — created, listed, labelled, and RBAC-controlled exactly like a Pod.

- **Structural schema** enforces required fields, types, ranges, enums — bad specs are rejected at admission
- **Defaults** fill omitted fields; **printer columns** enrich `kubectl get`
- `kubectl explain <kind>.spec` is generated from the schema — self-documenting

Without a schema, a CR is an untyped blob. *With* one, it gets the same guardrails as native objects.



CRDs + Controllers = Operators

A definition is data; a controller makes it real

The Reconciliation Loop

A controller continuously...

1. **Watches** custom resources of its type
2. **Compares** desired spec to actual cluster state
3. **Acts** to close the gap (create Deployments, call APIs...)
4. **Reports** progress in the resource's status

This is the same control-loop pattern Kubernetes uses for built-in objects — an **operator** just applies it to your domain.

You've Been Using Operators All Week

Custom Resource	The controller reconciles it into...
Flux HelmRelease	an installed/upgraded Helm chart
cert-manager Certificate	an issued & auto-renewed TLS secret
Gateway API HTTPRoute	programmed routes on the Envoy Gateway
Kyverno ClusterPolicy	admission rules enforced on every request

Building the controller is the operator pattern (frameworks: Kubebuilder, Operator SDK) — a subject of its own.

Lab: Custom Resource Definitions

Hands-on: add your own resource type

- Register a Website **CRD** and watch it appear in `api-resources`
- Create **custom resources** and view them via printer columns
- Trip the **OpenAPI schema** with an invalid spec
- Discover the type with `kubectl explain` and short names

Duration: ~25-35 minutes

Key Takeaways

- A **CRD** teaches the API server a new kind — extending Kubernetes with no restart or recompile.
- **Custom resources** behave like built-ins: same verbs, labels, RBAC, and output formats.
- An **OpenAPI schema** validates specs; printer columns, short names, and explain make the type discoverable.
- A CRD is only the **API**; a **controller/operator** reconciles custom resources into real state.
- Flux, cert-manager, Gateway API, and Kyverno are all CRDs + controllers — now you can read them.